HACKATÓN LINCECODE 2026 / UAdeO

ACCESO LINCE

Control de accesos y aforo universitario en tiempo real

Memoria técnica del prototipo, análisis, diseño, implementación y evidencias

Universidad Autónoma de Occidente · Unidad Mazatlán

Equipo desarrollador: _____

Fecha de elaboración: 8 de noviembre de 2026

Resumen ejecutivo

ACCESO LINCE es un prototipo académico para registrar y supervisar la entrada y salida de personas en un campus universitario. Responde al reto de reducir la dependencia de controles manuales mediante una **web institucional**, una **aplicación Android de vigilancia** y una **base de datos compartida**.

El registro por credencial QR permite validar usuarios activos, evitar lecturas duplicadas, registrar movimientos y consultar la ocupación estimada por categoría, carrera o grupo. Dirección y Administración tienen funciones diferenciadas; el alumnado consulta únicamente su información personal.

Componente	Tecnología	Propósito
Interfaz institucional	HTML, CSS, JavaScript	Administración, alumnos, métricas y reportes
Servidor / API	Node.js 22	Autenticación, validación, roles y reglas de negocio
Datos	SQLite (WAL)	Personas, presencia, eventos, cuentas, turnos y auditoría
App vigilante	Android nativo / Java	Cámara continua, QR, turnos y registro exprés
Escaneo	CameraX + ML Kit	Lectura de códigos QR y barras
Actualización web	Server-Sent Events	Refrescar indicadores tras los movimientos

Ruta de lectura

01 Problema y alcance

02 Objetivos, roles y responsabilidades

03 Requisitos de convocatoria y cumplimiento

04 Arquitectura y modelo de datos

05 Flujo y módulos funcionales

06 Código, pruebas y evidencias

07 Uso documentado de IA y conclusiones

Alcance del documento: memoria técnica. El manual operacional con instrucciones paso a paso se elaborará y entregará por separado.

1. Planteamiento del problema

En una universidad con distintas puertas y un flujo constante de alumnado, personal y personas externas, la supervisión puramente visual dificulta conocer quién ha registrado su ingreso o egreso. La administración necesita información consultable, pero el personal de vigilancia requiere una herramienta rápida que no interrumpa el paso.

El reto plantea controlar accesos mediante credenciales institucionales, conservar eventos históricos y construir un tablero de aforo en tiempo real. También contempla eventos extraordinarios, visitantes y personal de proveedores.

Justificación de la propuesta

- Disminuir el registro manual y los errores de transcripción en puertas de acceso.
- Contar con una bitácora histórica que no pueda alterarse mediante operaciones ordinarias.
- Concentrar la información en un servidor, aun cuando operen varios teléfonos Android.
- Brindar a Dirección y Administración indicadores útiles para supervisión y decisiones.
- Hacer más ágil la experiencia del vigilante mediante escaneo continuo y acciones cortas.

Alcance

Incluye el prototipo	Queda fuera de la demostración validada
Inicio de sesión según rol, catálogo de usuarios, QR y control de acceso.	Identificación biométrica o validación oficial de documentos.
Historial, aforo estimado, importación masiva y reportes.	Confirmación física de ubicación: el sistema depende del escaneo.
Turnos, relevos y vinculación de dispositivos autorizados.	Operación de producción con datos personales reales y HTTPS.
Registro fotográfico temporal de visitantes y proveedores.	Lectura OCR y autenticación automática de INE/gafetes.

Se trabaja con datos ficticios. La presencia calculada es una estimación administrativa basada en entradas y salidas registradas; no equivale a un censo físico infalible.

2. Objetivos y organización

Objetivo general

Diseñar y desarrollar un sistema funcional que **registre accesos universitarios por QR** y permita supervisar el aforo y la actividad del campus mediante una web administrativa y una aplicación Android sincronizadas con una misma base de datos.

Objetivos específicos

- Crear un catálogo de estudiantes, personal y visitantes con identificador, estado y credencial QR.
- Implementar validaciones de credenciales, alternancia entrada/salida y prevención de rebotes.
- Conservar movimientos con fecha, hora, puerta y responsable de operación.
- Presentar indicadores de aforo, historia individual y reportes filtrables.
- Organizar cuentas y permisos de Dirección, Administración, alumnado y vigilancia.
- Permitir varios vigilantes y dispositivos simultáneos, con turnos y relevos auditables.
- Facilitar altas masivas por Excel y el registro exprés de personas externas.

Roles del sistema

Rol	Funciones principales
Dirección	Visión general; administración de alumnos, guardias y administradores; supervisión de movimientos y dispositivos.
Administración	Gestión de alumnos, guardias, visitantes, reportes y turnos; sin facultad de crear administradores.
Alumno	Acceso web restringido a su perfil, credencial QR, calendario e historial propio.
Vigilancia	App Android: leer códigos, registrar accesos y operar turnos/relevos; sin facultades directivas.
Personas externas	Registro limitado de visitante o proveedor y seguimiento de movimiento.

Participación del equipo y uso de IA

El equipo definió el problema, evaluó los requisitos, tomó decisiones de producto, eligió la identidad visual, realizó pruebas y retroalimentó las versiones. **ChatGPT (GPT-6)** se utilizó como asistencia para analizar requisitos, proponer diseños, generar y revisar código, elaborar pruebas y documentar el desarrollo. Las decisiones y resultados se verificaron de manera iterativa por el equipo.

3. Requisitos del reto

Correspondencia entre los puntos de la convocatoria proporcionada y la solución. La clasificación diferencia lo implementado de las extensiones pendientes.

Requisito	Estado	Observación
Catálogo con fotografía y estado	Implementado	Fotografía opcional, datos y estatus activo/inactivo.
Historial inmutable y reportes	Implementado	Eventos protegidos por disparadores SQL y consultas históricas.
Validación alternada entrada/salida	Implementado	Según presencia actual y reglas de vigencia.
Cierre automático de medianoche	Alternativa implementada	Consolidación a las 23:59 sin registrar egresos ficticios.
Aforo por grupo / carrera	Implementado	A partir de eventos y estado de presencia.
Avisos al tutor por correo/WhatsApp	Pendiente	Requiere consentimiento y servicio de mensajería.
Cámara para captura y respuesta visual	Implementado	App nativa, lectura continua y alertas operativas.
Interfaz kiosco/pantalla completa	Adaptado	Se eligió app Android de vigilancia más web de gestión.
Panel responsive administrativo	Implementado	Interfaz web adaptable y varias paletas.
Nivel 3: IA de anomalías o chat	Pendiente	No se presenta como módulo activo del sistema.

Nota de diseño: no se generan salidas automáticas falsas. A las 23:59 se conserva quién no registró egreso, una decisión pensada para proteger la integridad de los hechos registrados.

4. Arquitectura del sistema

Se adoptó una arquitectura cliente-servidor con **un origen de datos común**. La web se ejecuta en el navegador y Android consume las rutas de la API. El servidor valida las credenciales y escribe cada movimiento en SQLite.

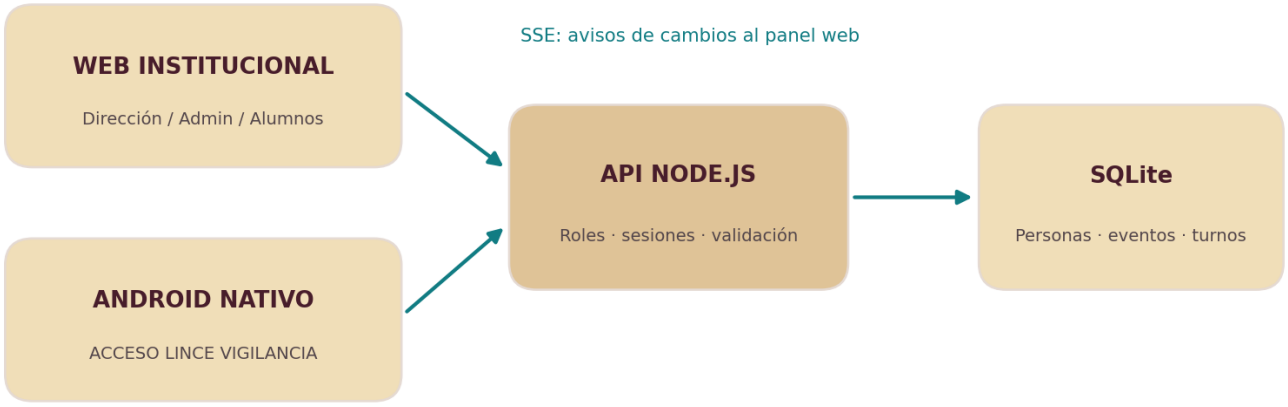


Figura 1. Arquitectura lógica simplificada. Las dos interfaces comparten validación y persistencia en el servidor.

Componentes y comunicación

Web. Pantallas para Dirección, Administración y alumnos, además de informes y gestión de dispositivos.

Android. Aplicación nativa denominada ACCESO LINCE VIGILANCIA; la cámara permanece activa durante la lectura, salvo al pausarse.

API. Sesiones, validación de roles, entradas/salidas, importación, turnos y fotografías efímeras.

Persistencia. SQLite con claves foráneas, WAL y tablas de auditoría; las transacciones protegen operaciones concurrentes.

Actualización. Server-Sent Events (SSE) informa de cambios al tablero; el teléfono utiliza respuestas de la API.

Decisiones tecnológicas

Se eligieron Node.js y SQLite por su facilidad de instalación local y capacidad de ejecutar la demostración en una computadora. Android nativo con CameraX y ML Kit resuelve las limitaciones de la cámara web móvil sobre HTTP en una red local.

5. Modelo de datos

La estructura de datos separa el **estado actual** de cada persona de la **historia de eventos**. Así se puede calcular el aforo sin sobrescribir la evidencia de movimientos anteriores.

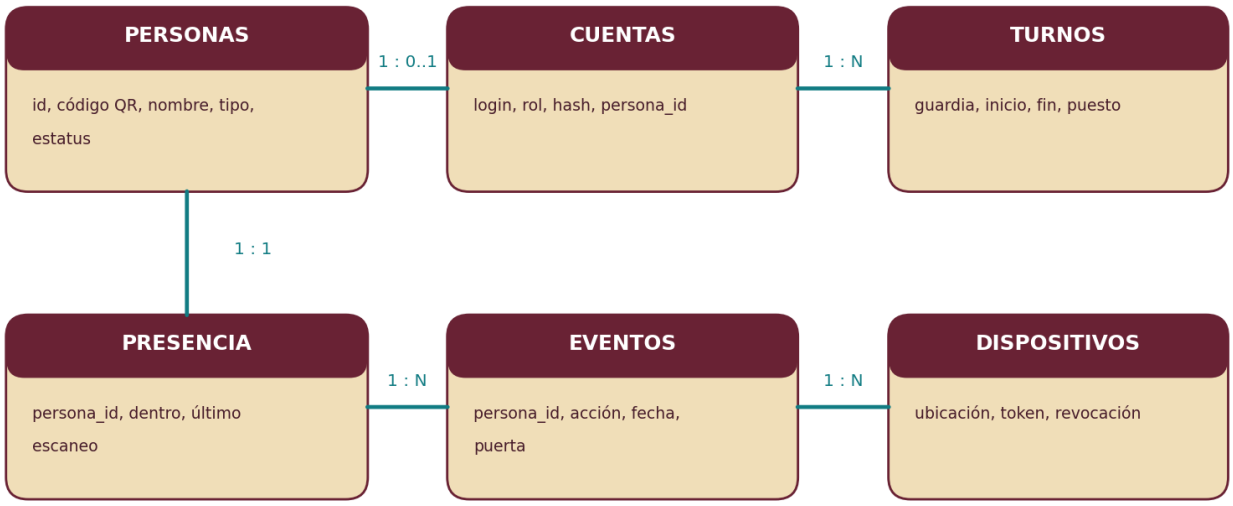


Figura 2. Relaciones principales simplificadas a partir del esquema SQLite de la versión V0.5.2.

Entidades adicionales

El esquema incluye también **cierres diarios** (`day_closures`), **eventos especiales** y sus cambios, **correcciones auditadas**, **rotación de QR**, **relevos** (`guard_handovers`), **vinculaciones temporales** (`guard_device_pairs`) e **ingresos exprés** (`express_entries`).

Integridad y trazabilidad

- Los eventos de entrada/salida se agregan a `access_events`; disparadores impiden UPDATE y DELETE ordinarios.
- `presence` sí cambia porque resume el último estado registrado, no sustituye la bitácora.
- Las cuentas usan hash derivado con `scrypt` y sal por usuario.
- Los dispositivos pueden revocarse, y los códigos de vinculación son temporales y de un solo uso.
- Fotografías de registro exprés: cifrado AES-256-GCM y vencimiento previsto a 24 horas.

6. Flujo de validación QR

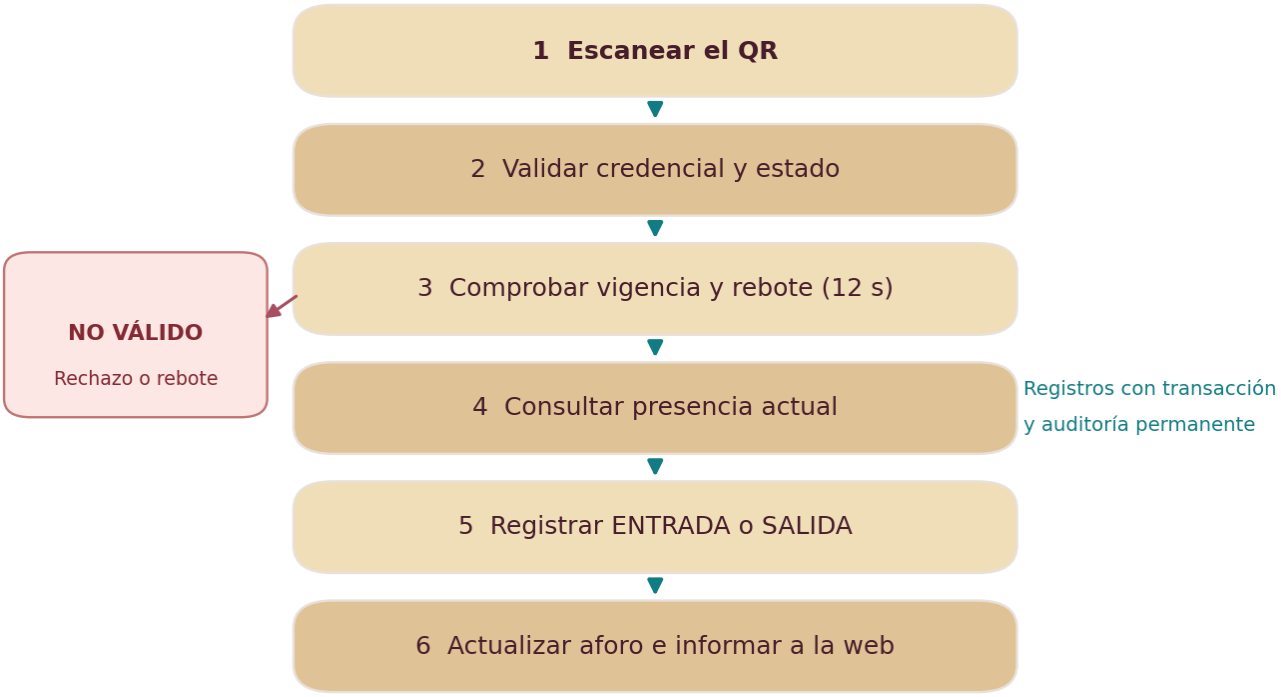


Figura 3. Diagrama del acceso habitual: validación, rebote, movimiento y actualización del tablero.

Caso de uso: ingreso de estudiante

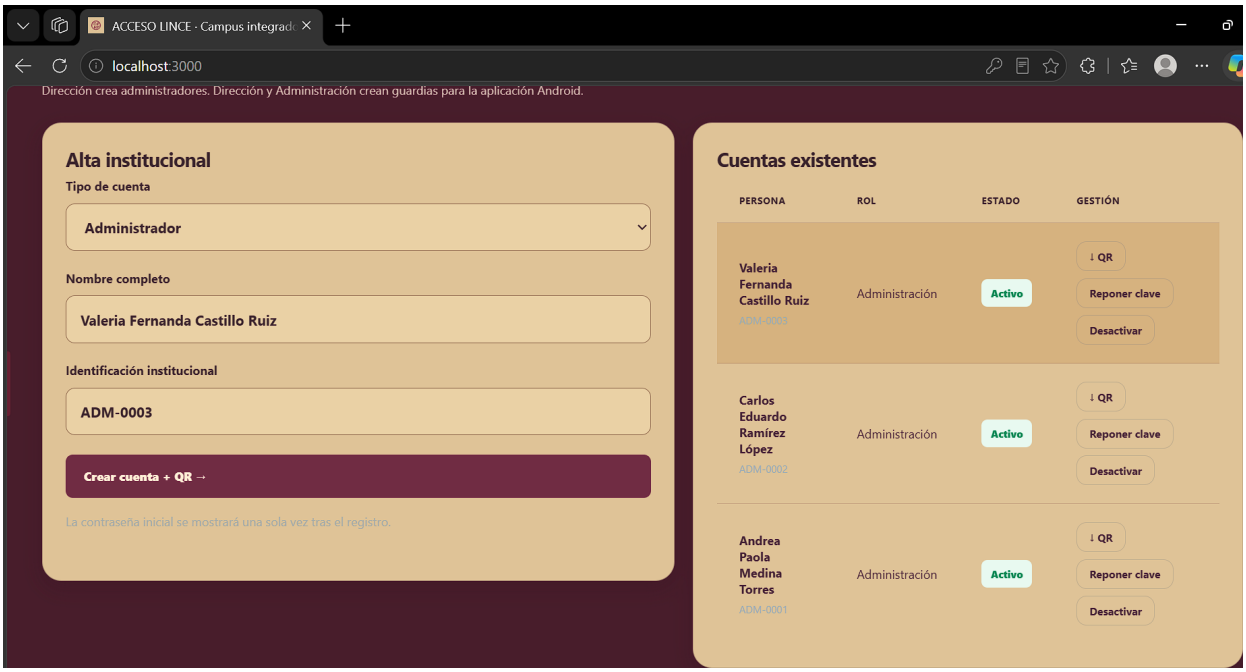
Precondición: el alumno tiene una credencial QR vigente y activa. El Android está vinculado y existe un vigilante autenticado.

Secuencia: cámara lee el código → la API comprueba usuario, vigencia y última lectura → registra una entrada si la persona estaba fuera (o salida si estaba dentro) → actualiza presencia y comunica el cambio al panel.

Casos excepcionales: código inexistente, cuenta inactiva, autorización vencida, rebote de 12 segundos o error de conectividad. La aplicación presenta una respuesta y no inventa un registro exitoso.

7. Web institucional

La interfaz utiliza la identidad visual de UAdeO: guinda institucional, paneles arena/champán, tipografía legible y un menú lateral por secciones. La portada y los módulos muestran el nombre **ACCESO LINCE**.



Evidencia A. Gestión de cuentas desde Dirección: alta de administrador, estado activo y controles de gestión. Datos mostrados: ficticios.



Evidencia B. Sección de Androids autorizados: nombre del equipo, puerta asignada y generación de vinculación temporal.

Capturas del prototipo durante las pruebas de noviembre de 2026. No se incluyen contraseñas ni códigos de vinculación activos.

8. App y módulos operativos

ACCESO LINCE VIGILANCIA es la aplicación nativa Android. Usa los colores y el logotipo institucional de la web. Fue diseñada para que el guardia trabaje con pocos pasos y pueda alternar de operador en un dispositivo autorizado.

Módulo	Proceso previsto o implementado
Acceso de vigilante	Gafete QR del guardia activo + dispositivo vinculado; sin PIN en el flujo acordado.
Escáner	Lectura continua, resultados de entrada/salida, advertencia de repetidos y opción manual de respaldo.
Turnos	Consultar turnos asignados, comenzar, finalizar, ofrecer y confirmar relevos.
Visitantes y proveedores	Captura fotográfica de documento de prueba, revisión del original y registro de entrada.
Perfil personal	Datos del vigilante, QR y gafete en PDF.
Multidispositivo	Varias tabletas/teléfonos, cada una con sesión y puerta vinculada; servidor común.

Límites de identificación

Un QR estático puede reproducirse si se fotografía. El dispositivo vinculado reduce el alcance, pero no elimina la suplantación. Para la demostración se emplean datos ficticios; el despliegue institucional requiere autenticación más robusta, HTTPS y lineamientos de privacidad.

Uso de fotografías

La captura exprés no confirma automáticamente la identidad: el vigilante revisa el documento y autoriza su registro. No se integra OCR. La fotografía se cifra en el servidor y se programa su eliminación a las 24 horas; queda la bitácora del ingreso.

Evolución del diseño

Se sustituyó un lector web móvil dependiente de permisos del navegador por Android nativo para proporcionar escaneo continuo. Posteriormente se amplió la solución con gafetes para vigilantes, varios dispositivos y acciones de turno.

9. Código fuente y funcionamiento

El software se organiza por responsabilidad, de manera que el código de visualización no administra directamente la base de datos. A continuación se presentan fragmentos representativos y rutas de archivos del proyecto.

Protección del historial con SQLite

```
CREATE TRIGGER IF NOT EXISTS immutable_events_update
BEFORE UPDATE ON access_events
BEGIN SELECT RAISE(ABORT, 'El historial es inmutable'); END;

CREATE TRIGGER IF NOT EXISTS immutable_events_delete
BEFORE DELETE ON access_events
BEGIN SELECT RAISE(ABORT, 'El historial es inmutable'); END;
```

Explicación: los eventos se registran como historial anexado; un usuario ordinario no modifica ni elimina acciones ya almacenadas. Archivo: server.js.

Transacciones y rebote de escaneo

```
const REBOUND_MS = 12000;
// El servidor aplica una transacción indivisible.
const outcome = runTransaction(() => {
  const p = personByCode(code);
  if (!p) return { ok:false };
  if (p.last_scan_at &&
      at.getTime()-Date.parse(p.last_scan_at)<REBOUND_MS)
    return { ok:false, rebound:true };
  const action = p.inside ? 'salida' : 'entrada';
  // Insertar evento y actualizar presencia.
});
```

Explicación: dos teléfonos que envían una lectura casi simultánea no deben generar un movimiento duplicado. El estado de presencia determina la operación normal. Fragmento condensado con fines explicativos; véase server.js para implementación completa.

Organización de archivos

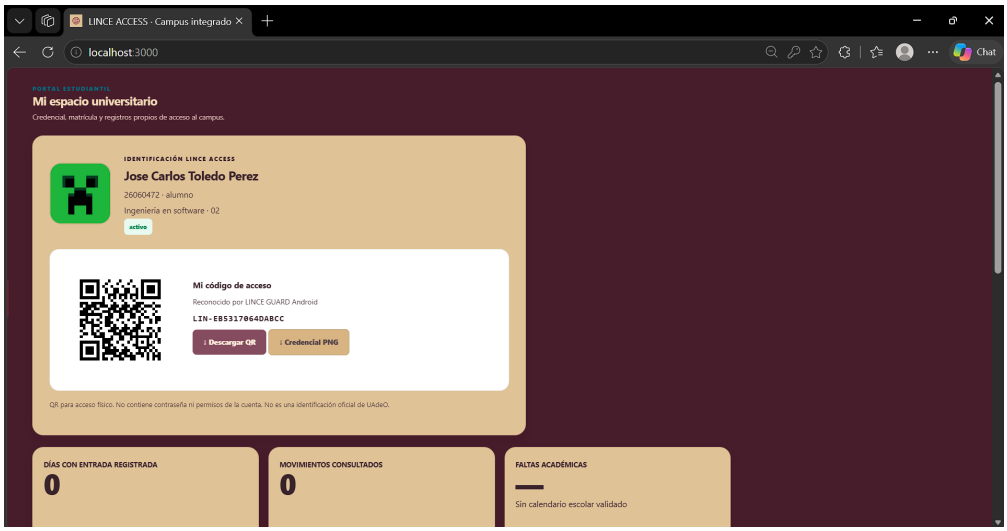
Archivo / módulo	Responsabilidad
server.js	Servidor, SQLite, API de escaneo y métricas
v05.js / bulk-import.js	Cuentas, permisos, QR e importación Excel
shifts.js	Turnos, horarios y relevos
guard-operations.js	Dispositivos autorizados y registros exprés
public/	Interfaz web y temas institucionales
Android: MainActivity / ApiClient	Pantallas y comunicación con la API
Android: ScanGate / ScanQueue	Filtro y cola de escaneos continuos

10. Pruebas y evidencias

Se efectuaron pruebas automáticas sobre bases temporales y pruebas manuales de navegación y escaneo durante el desarrollo. La evidencia se clasifica según el origen para evitar confundir pruebas unitarias con una validación integral en condiciones de producción.

Evidencia	Resultado documentado
Integración servidor, web y roles	Batería npm test con 10 módulos; sin fallos en la ejecución registrada.
Escaneo continuo, rebotes y cola	26 verificaciones Java de ScanGate/ScanQueue superadas.
V0.5.1 turnos y relevos	46 comprobaciones de roles, horarios, relevos y auditoría.
V0.5.2 operaciones móviles	64 verificaciones de dispositivos, QR, fotografías, privacidad y concurrencia.
Migración de datos	9 verificaciones de integridad en el paso V0.5.1 → V0.5.2.
Carga masiva	Prueba automatizada de hasta 500 alumnos ficticios; Excel de 100 preparado para demostración.
Instalación limpia	Primer arranque crea Dirección, deja tablas vacías y conserva datos en reinicios.
Android instalable	Compilación APK obtenida; desempeño de la versión final en cada teléfono debe comprobarse en vivo.

Evidencias visuales del desarrollo



Evidencia C. Vista de perfil estudiantil de prueba y su QR personal, generados por la web. La imagen es ficticia.

Para el evento se recomienda añadir fotografías nuevas de la demostración en vivo: teléfono leyendo QR, contador antes/después y bitácora del movimiento. No se adjuntan capturas con contraseñas o documentos reales.

11. Asistencia con IA: metodología

Herramienta utilizada: ChatGPT, modelo GPT-6. El equipo la empleó como apoyo conversacional durante el análisis, construcción, revisión y documentación del prototipo. Las instrucciones se emitieron iterativamente con cambios de alcance y pruebas sobre el proyecto.

La siguiente cronología es una **selección de prompts relevantes** y objetivos técnicos recuperados de la conversación y del archivo de seguimiento del proyecto. Las frases entre comillas son extractos textuales breves cuando están disponibles; otras entradas son resúmenes de la instrucción, no transcripciones literales.

7 de noviembre · Inicio y primeras versiones

Etapa	Instrucción del participante / finalidad	Resultado
Idea inicial	“Bro se viene el hackaton en mi universidad vamos a empezar tu y yo contra todos”.	Lectura de convocatoria, requisitos y prioridades.
Identidad	“colores tipo uadeo”.	Paleta y logo institucional para web.
Accesibilidad	“hay que pensar en la accesibilidad para los guardias” (extracto).	Controles simples, tamaño y alertas visibles.
Arquitectura	“...la web para dirección y administración y la de guardias para dispositivos móviles o tablets” (extracto).	Separación de interfaces sobre una API compartida.
Cámara	“Necesitamos que escanee siempre en vivo” (extracto).	Transición a app Android con CameraX y ML Kit.
Conectividad	“...creé un nuevo alumno y puse su número y funcionó” (extracto).	Pruebas de alumno nuevo, QR y servidor central.

Uso de las respuestas de IA

Las respuestas de ChatGPT se utilizaron como propuestas de arquitectura, código JavaScript/Java, interfaz visual y pruebas. El equipo revisó la ejecución, indicó errores y aprobó los cambios. La IA no sustituyó la validación funcional en dispositivos o navegadores.

La convocatoria indica incluir todos los prompts cronológicamente. Esta selección resumida NO constituye la transcripción completa; si el jurado exige totalidad literal, deberá agregarse la exportación íntegra y ordenada del historial original.

12. Cronología de prompts clave

8 de noviembre · Consolidación web y Android

Tema	Prompt o instrucción representativa	Decisión o resultado
Visual web	Unificar el inicio de sesión con la paleta interna; sustituir “LINCE ACCESS” por “ACCESO LINCE” y eliminar la referencia al hackatón.	Identidad coherente en toda la web.
Alcance futuro	Conservar mejoras sugeridas para versiones posteriores y concentrarse en Android.	Prioridad al flujo de vigilancia.
Turnos	Incorporar perfiles, gafetes, turnos y relevos de guardias; supervisión desde la web.	Módulos web y API de turnos.
Login ágil	“...su inicio de sesión no lo hacemos con su gafete... sin tener que escribir su cuenta y su contraseña” (extracto).	Acceso operativo por QR y Android autorizado.
Concurrencia	“no solo será 1 guardia en turno y debe funcionar en diferentes dispositivos al mismo tiempo” (extracto).	Turnos y dispositivos simultáneos.
Externos	Registro exprés tomando foto de INE de visitante o gafete del proveedor (resumen).	API y captura fotográfica temporal.
Sin PIN	“no me gusta eso del pin ignora ese método de seguridad queremos agilidad” (extracto).	Flujo sin PIN; riesgo expresamente documentado.
Identidad APK	“usa ACCESO LINCE VIGILANCIA ... la misma paleta ... el mismo [logo] de la web” (extracto).	Nombre, iconografía y textos en español.
Datos de demo	“no sería mejor dejar la base de datos limpia” (extracto).	Instalación institucional vacía con alta inicial de Dirección.
Prueba académica	“ocupamos cargar estudiantes con excel de 100 alumnos diferentes carreras” (extracto).	Archivo ficticio multidisciplinario para importación.

Las paráfrasis no se presentan como citas textuales. Se conservaron los cambios de criterio solicitados por el equipo (por ejemplo, un dispositivo compartido → varios dispositivos simultáneos).

13. Análisis crítico y cierre

Aportaciones principales

- Un modelo de datos único evita discrepancias entre la web y varios Android.
- Escaneo continuo y bloqueo de rebotes de 12 segundos favorecen el flujo de entrada.
- Separación de roles limita acciones sensibles a cuentas directivas y administrativas.
- La importación masiva reduce el tiempo de registro de cientos de estudiantes ficticios.
- Los cierres diarios no falsifican salidas; la bitácora conserva movimientos originales.
- La interfaz institucional conserva el mismo nombre, logo y paleta en web y Android.

Limitaciones reconocidas

- La medición de aforo depende de que todas las personas escaneen al entrar y salir.
- La conexión local HTTP se limita a demostraciones; requiere HTTPS para operar con datos personales reales.
- El QR estático como inicio de guardia puede copiarse; el vínculo del dispositivo no prueba identidad física.
- Las notificaciones a tutores, la identificación automática de documentos y el módulo de IA de anomalías no están operativos.
- La robustez del escaneo final debe verificarse sobre dispositivos físicos y red de presentación.

Conclusión

ACCESO LINCE presenta un prototipo integrado y demostrable para control de accesos universitarios. La solución combina software administrativo, lectura QR Android, registros históricos y un servidor compartido. Su principal fortaleza es la integración entre decisiones operativas y datos verificables. El sistema **cumple el núcleo funcional de control de acceso y aforo estimado** y deja identificadas las tareas necesarias para una eventual implementación institucional.

Fuentes internas utilizadas para elaborar esta memoria: convocatoria fotografiada del Hackatón LinceCode 2026; código fuente ACCESO LINCE V0.5.2 (`server.js`, `v05.js`, `shifts.js`, `guard-operations.js`); proyecto Android ACCESO LINCE VIGILANCIA; `RESULTADOS_DE_PRUEBAS.txt`; `REGISTRO_IA.md`; capturas de ejecución proporcionadas por el equipo.

Anexo pendiente por decisión del equipo: manual operacional en documento independiente y, si se solicita cumplimiento literal, exportación cronológica íntegra de todos los prompts de IA.